

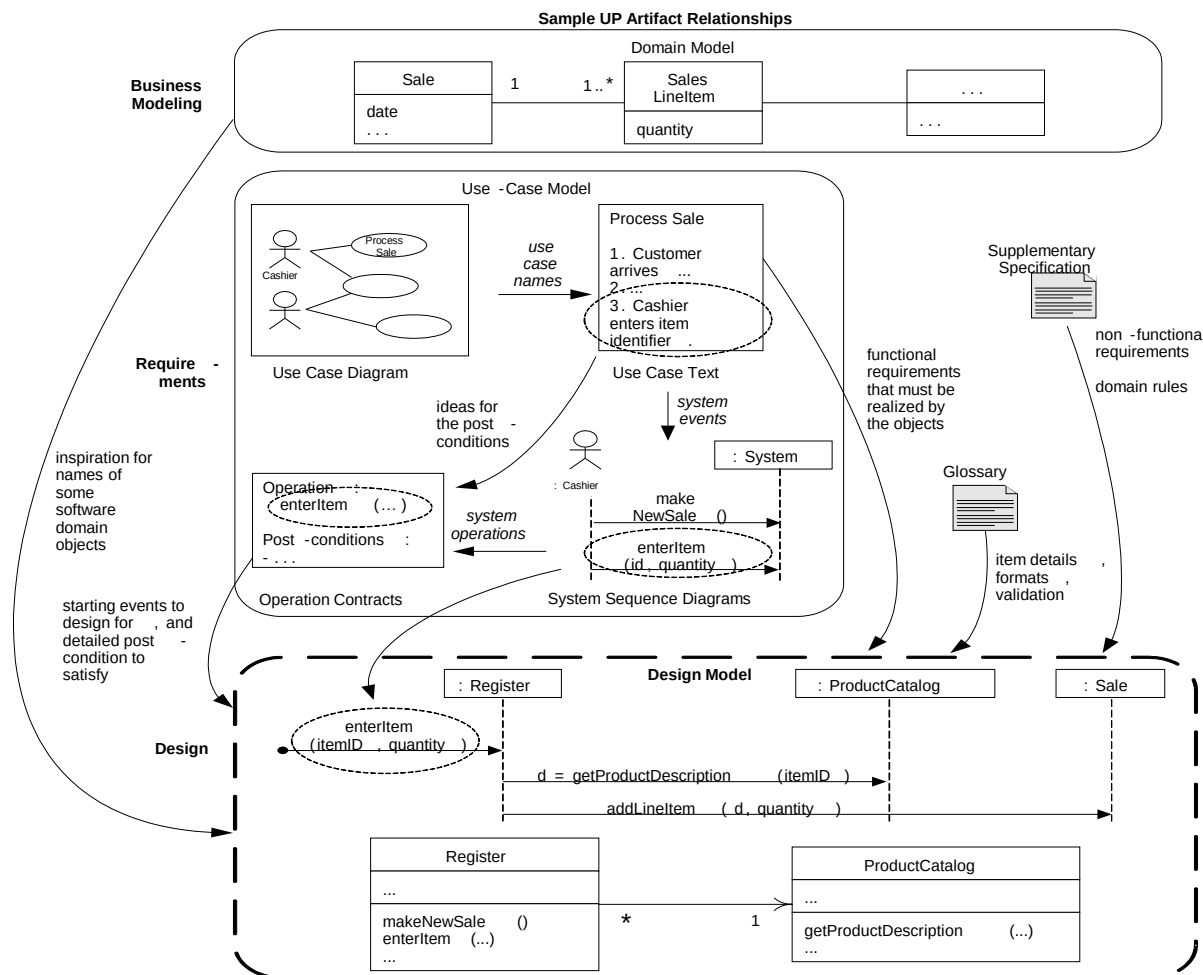
Chapter 18

Object Design Examples with GRASP



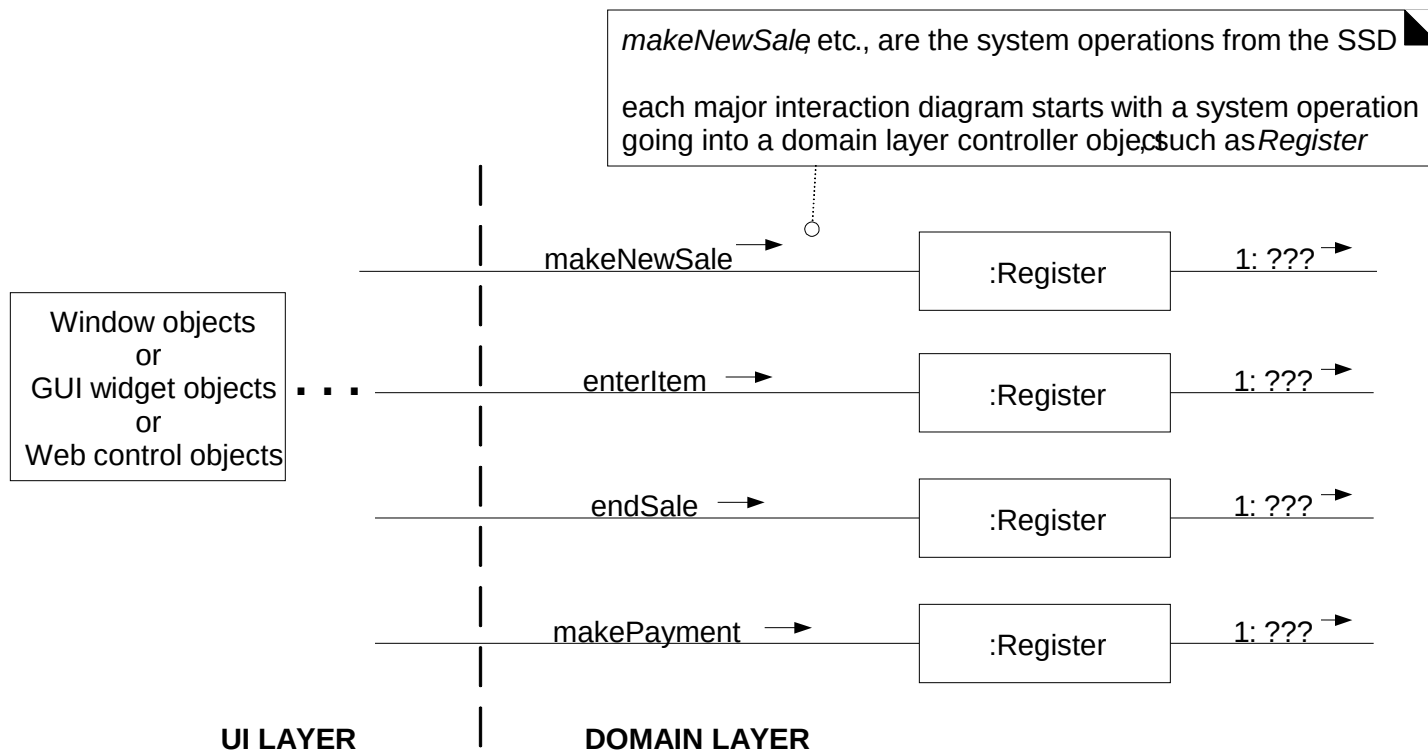
Use Case Realization

- It describes **how a particular use case is realized** within the **Design Model**, in terms of **collaborating objects**
- Designer can describe the **design of one or more scenarios** of a use case
 - each of these is called a use case realization



Use Case Realization

- The use case suggests the system operations that are shown in SSDs.
- The **system operations become the starting messages** entering the Controllers for domain layer interaction diagrams.

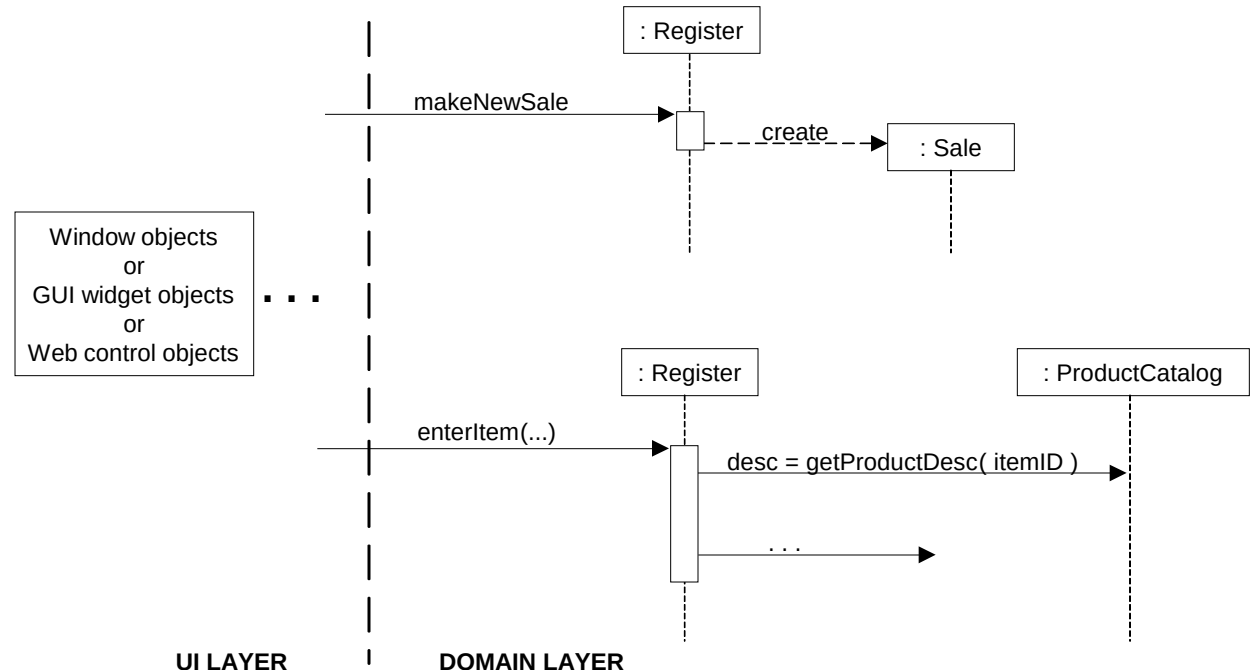


- Domain layer interaction diagrams illustrate how objects interact to fulfill the required tasks the use case realization.



System Operations in NextGen POS

- In the current iteration, we are considering scenarios and system operations identified on the SSDs of the **Process Sale use case**:
 - makeNewSale
 - enterItem
 - endSale
 - makePayment



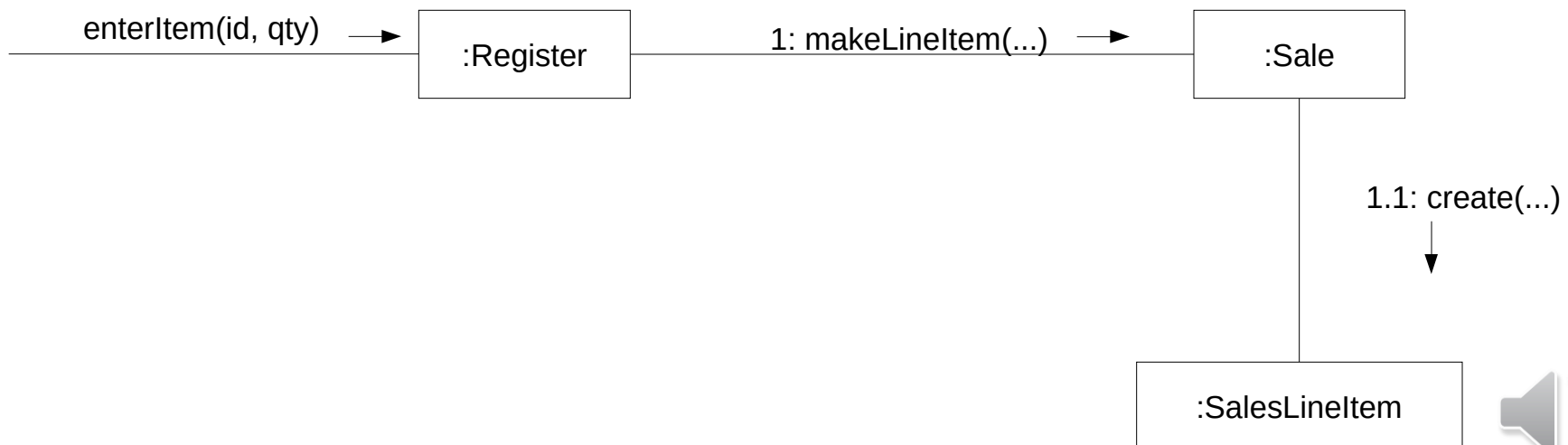
Key Point: The **system operations** in the **SSDs** are used as the **starting messages** into the **domain layer controller objects**.



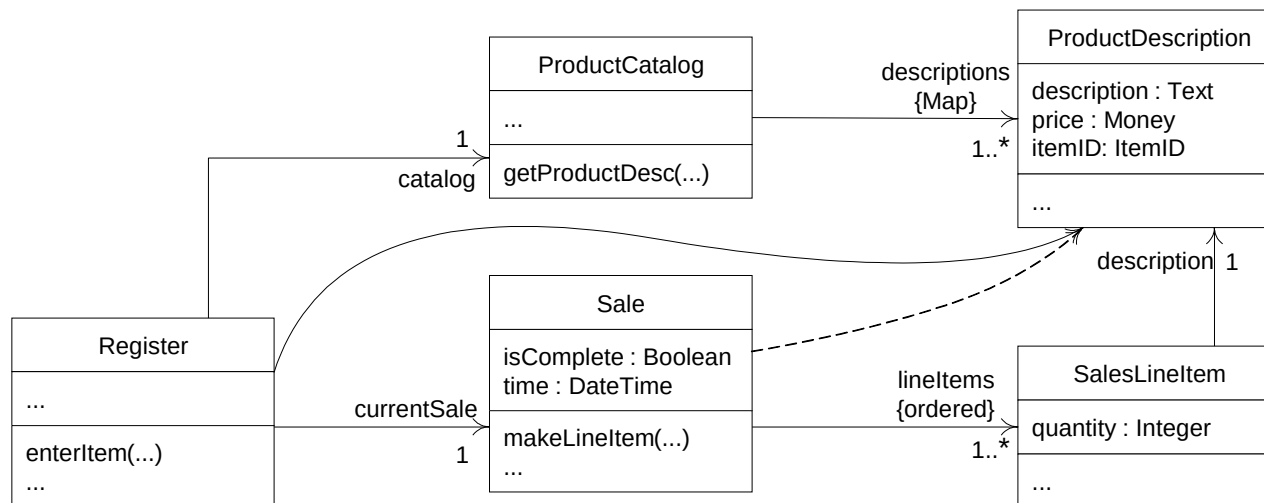
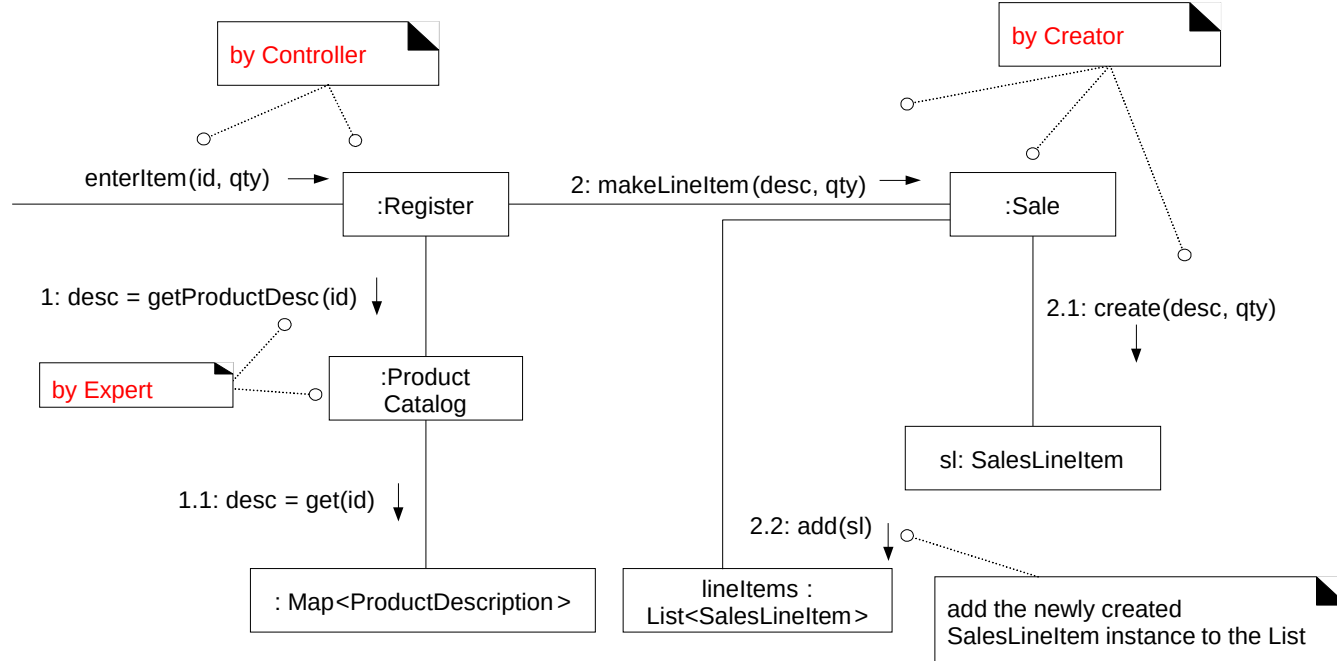
Operation Contract

- As discussed, use case realizations could be designed **directly from the use case text** or from one's domain knowledge.
- For some complex system operations, **operation contracts** may have been written that add more analysis detail.

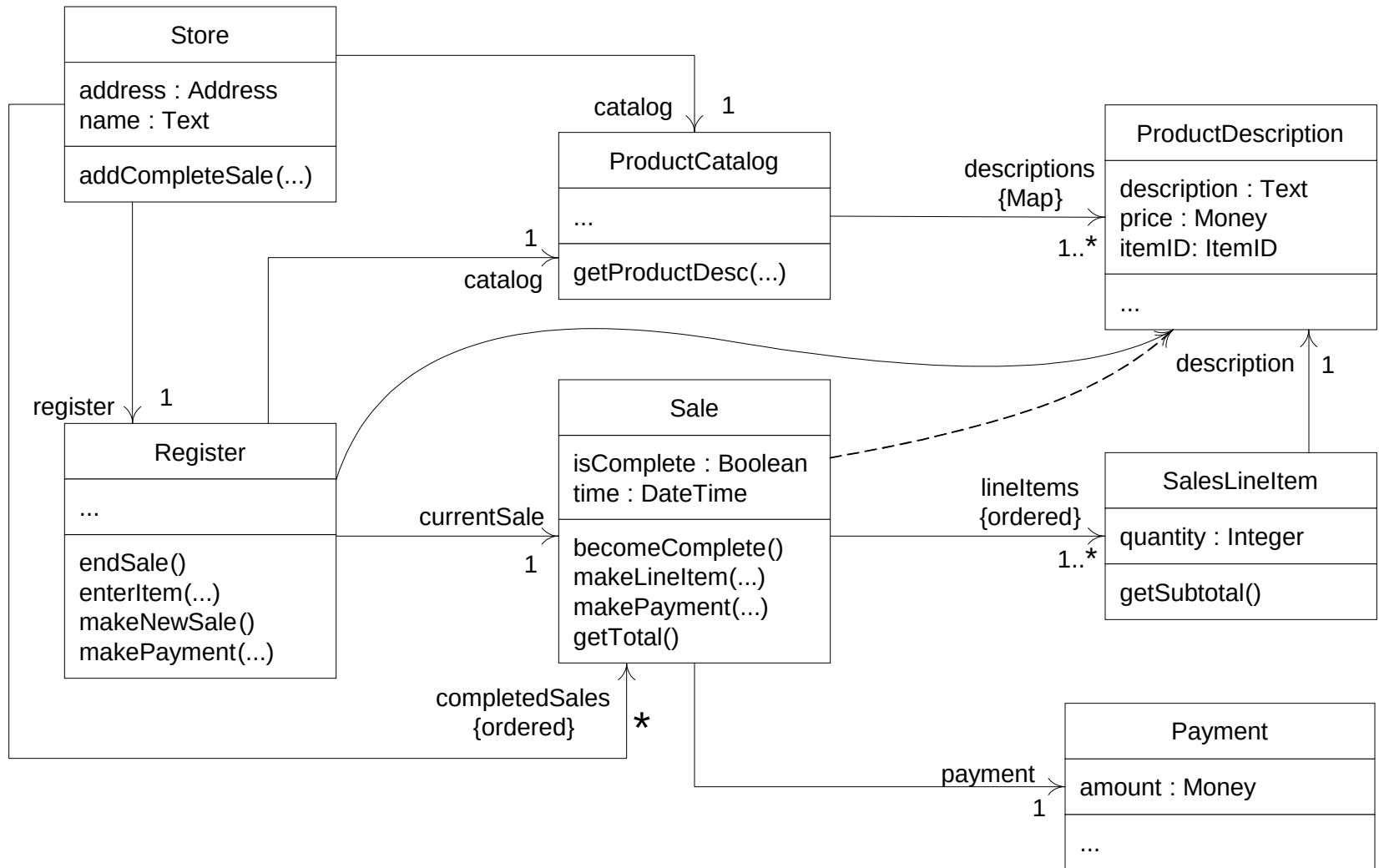
Operation:	enterItem(itemID: ItemID, quantity: integer)
Cross References:	Use Cases: Process Sale
Preconditions:	There is a sale underway.
Postconditions:	• A SalesLineItem instance sli was created (instance creation). • ...



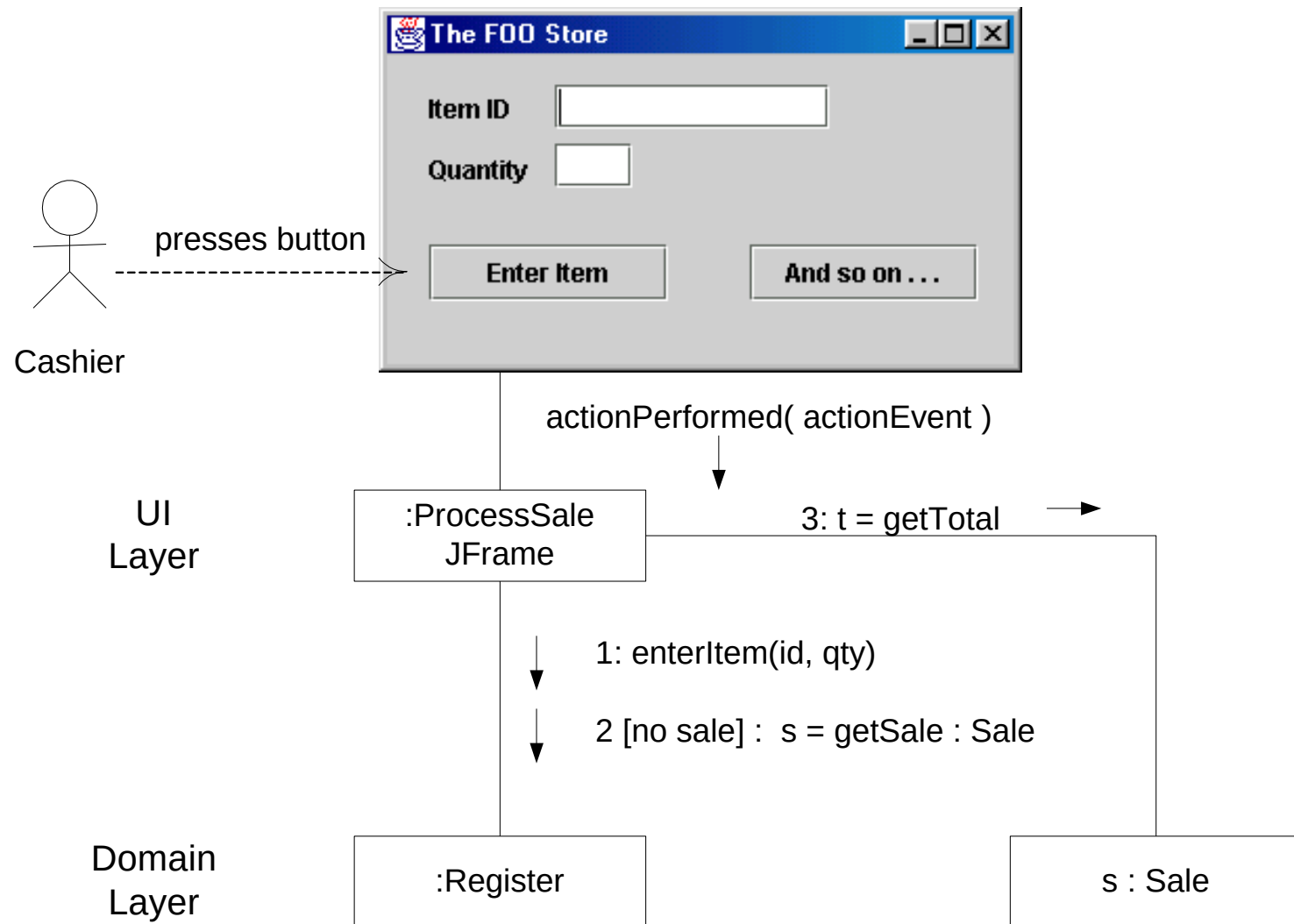
Design Models related with enterItem



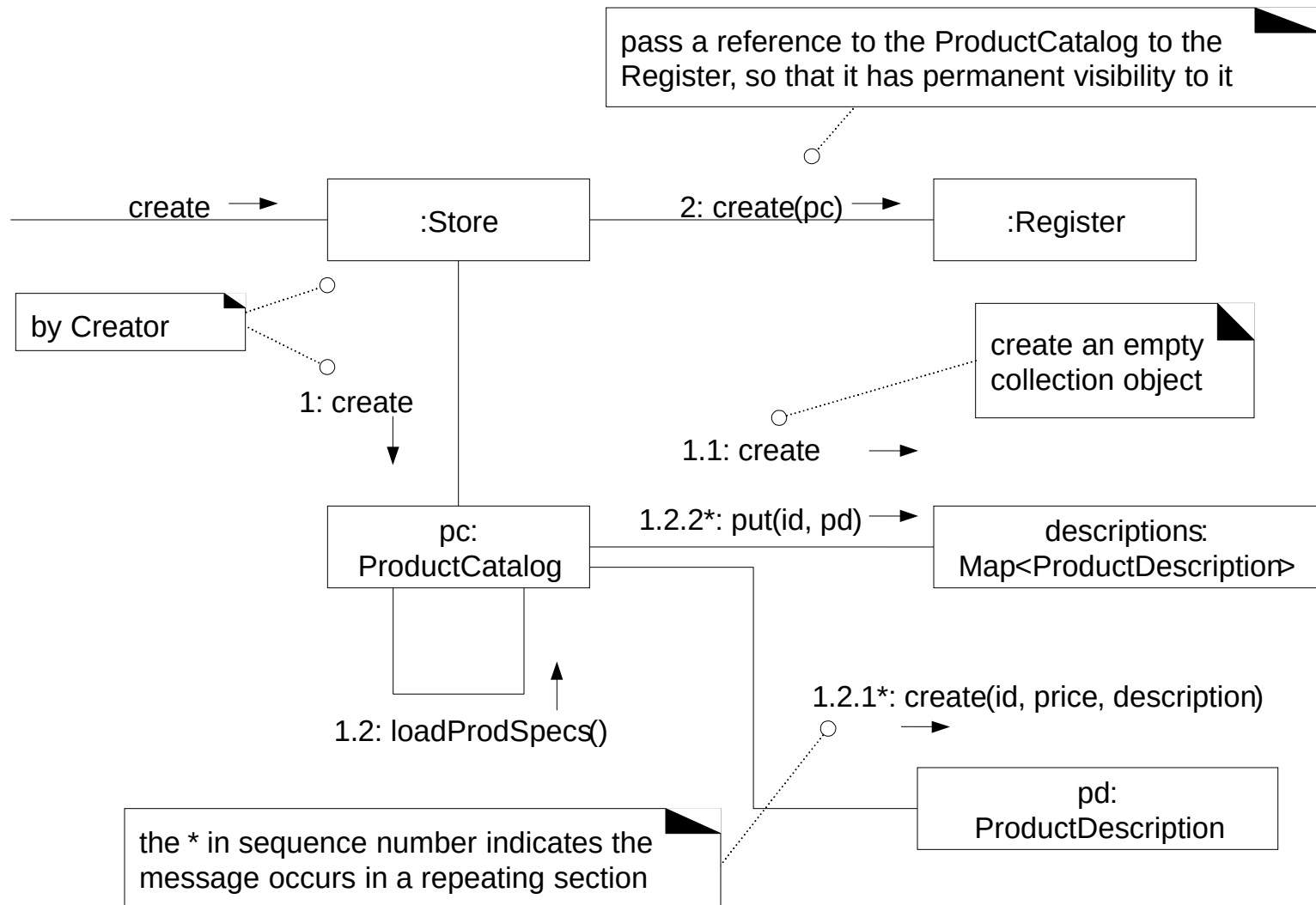
Final NextGen DCD for Iteration-1



Connecting the UI and Domain Layers



Creation of Initial Domain Object and Others



Chapter 19. Designing for Visibility



Visibility

- Ability of an object to "see" or have a reference to another object
- More generally, it is related to the **issue of scope**: Is one resource (such as an instance) within the scope of another?
- Visibility from object A to object B
 - Parameter visibility: B is a parameter of a method of A. → UML dependency
 - Local visibility: B is a (non-parameter) local object in a method of A. → UML dependency
 - Attribute visibility: B is an attribute of A. → UML association
 - Global visibility: B is in some way globally visible. (UML association)
- Motivation to consider visibility
 - For an object A to send a message to an object B, B must be visible to A

Chapter 20. Mapping Designs to Code

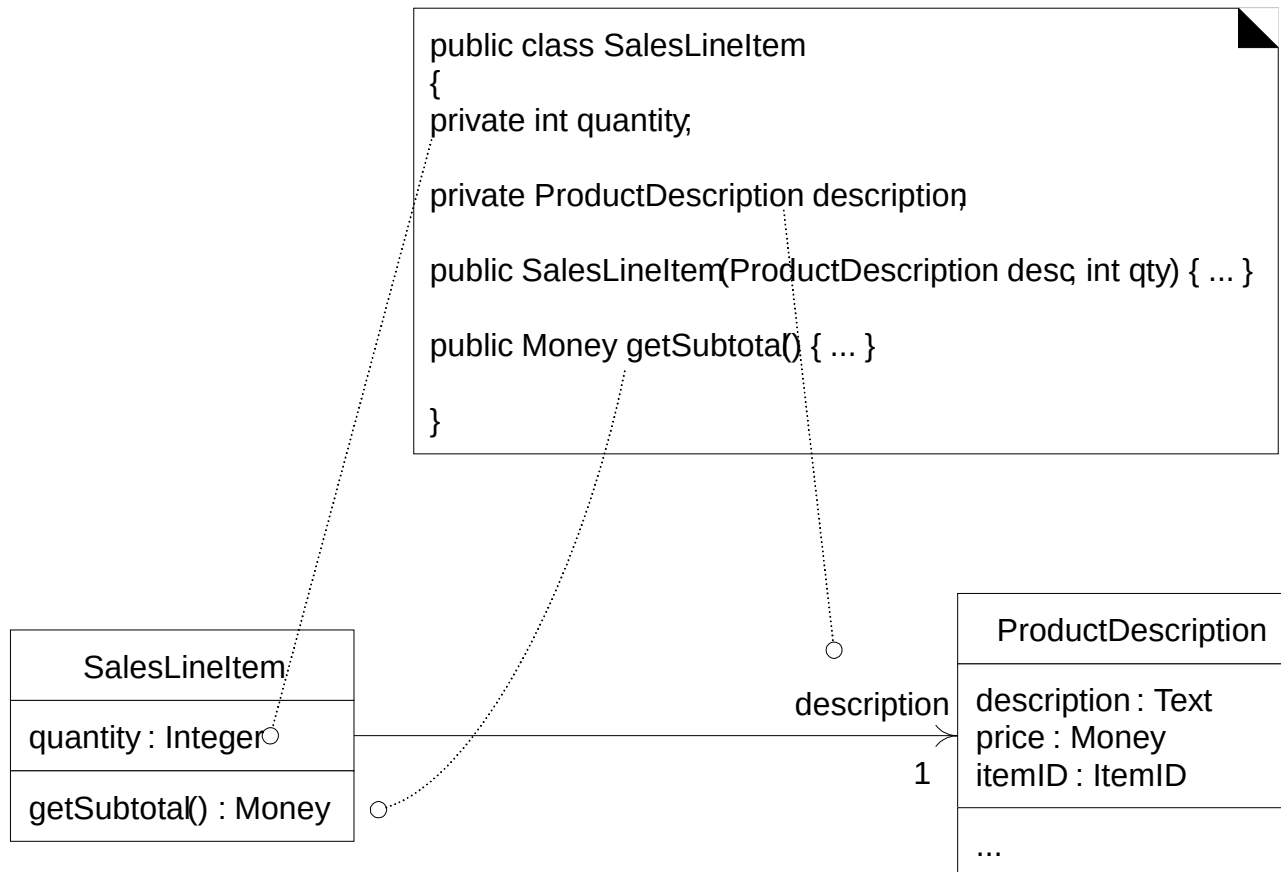


Mapping Designs to Code

- The UML artifacts created during the design work (**the interaction diagrams and DCDs**) will be used as input to **the code generation** process.
- **Implementation Model**
 - all the implementation artifacts:
 - source code, database definitions, JSP/XML/HTML pages, and so forth
- Implementation in an OO language requires writing source code for:
 - class and interface definitions
 - method definitions
- **Translation process from UML designs to code**
 - from **class diagrams** to **class definitions**,
 - from **interaction diagrams** to **method bodies**.

Creating Class Definitions from DCDs

- DCDs are sufficient to create a basic class definition in an OO language.
 - For example,
 - From the DCD, a mapping to the attribute definitions (Java fields) and method signatures for the Java definition (*SalesLineItem*) is straightforward.



Creating Methods from Interaction Diagrams

- The sequence of the messages in an interaction diagram translates to a series of statements in the method definitions.
 - For example,
 - The *enterItem* interaction diagram illustrates the Java definition of the *enterItem* method.

(Method) The *enterItem* message is sent to a *Register* instance; therefore, the *enterItem* method is defined in class *Register*.

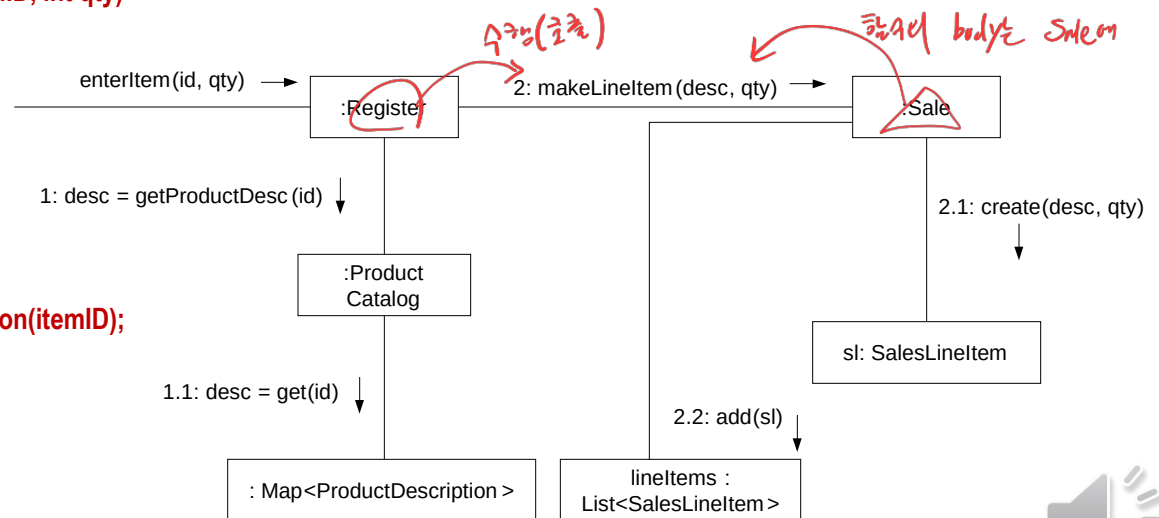
Message 2: The *makeLineItem* message is sent to the *Sale*.

public void enterItem(ItemID itemID, int qty)

currentSale.makeLineItem(desc, qty);

Message 1: A *getProductDescription* message is sent to the *ProductCatalog* to retrieve a *ProductDescription*.

ProductDescription desc = catalog.getProductDescription(itemID);



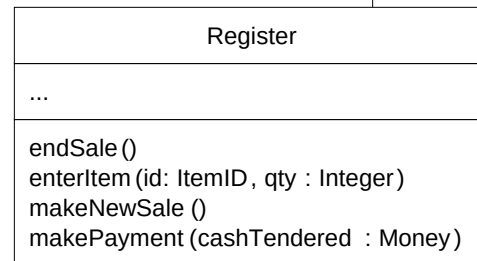
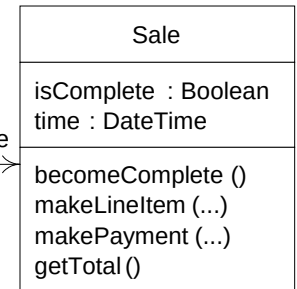
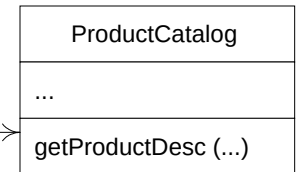
```

public class Register
{
    private ProductCatalog catalog ;
    private Sale currentSale ;

    public Register (ProductCatalog pc ) {...}

    public void endSale () {...}
    public void enterItem (ItemID id , int qty) {...}
    public void makeNewSale () {...}
    public void makePayment (Money cashTendered ) {...}
}

```

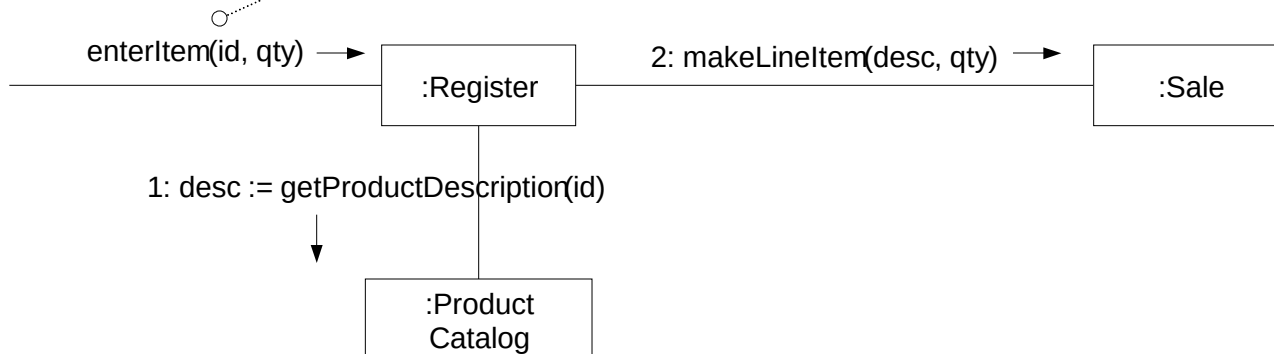


enterItem()

```

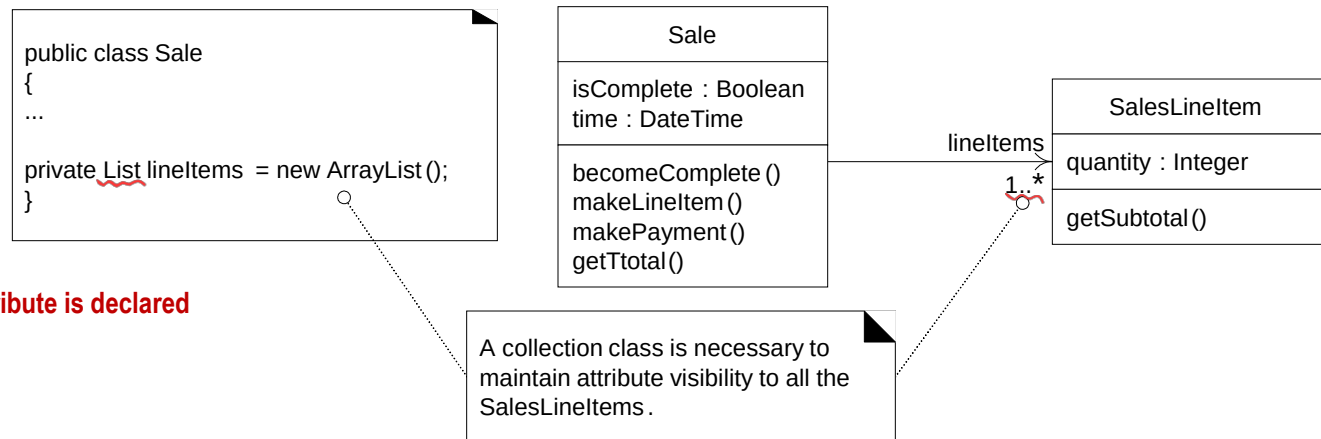
{
    ProductDescription desc= catalog.ProductDescription(id);
    currentSale.makeLineItem(desc, qty);
}

```



Collection Classes in Code

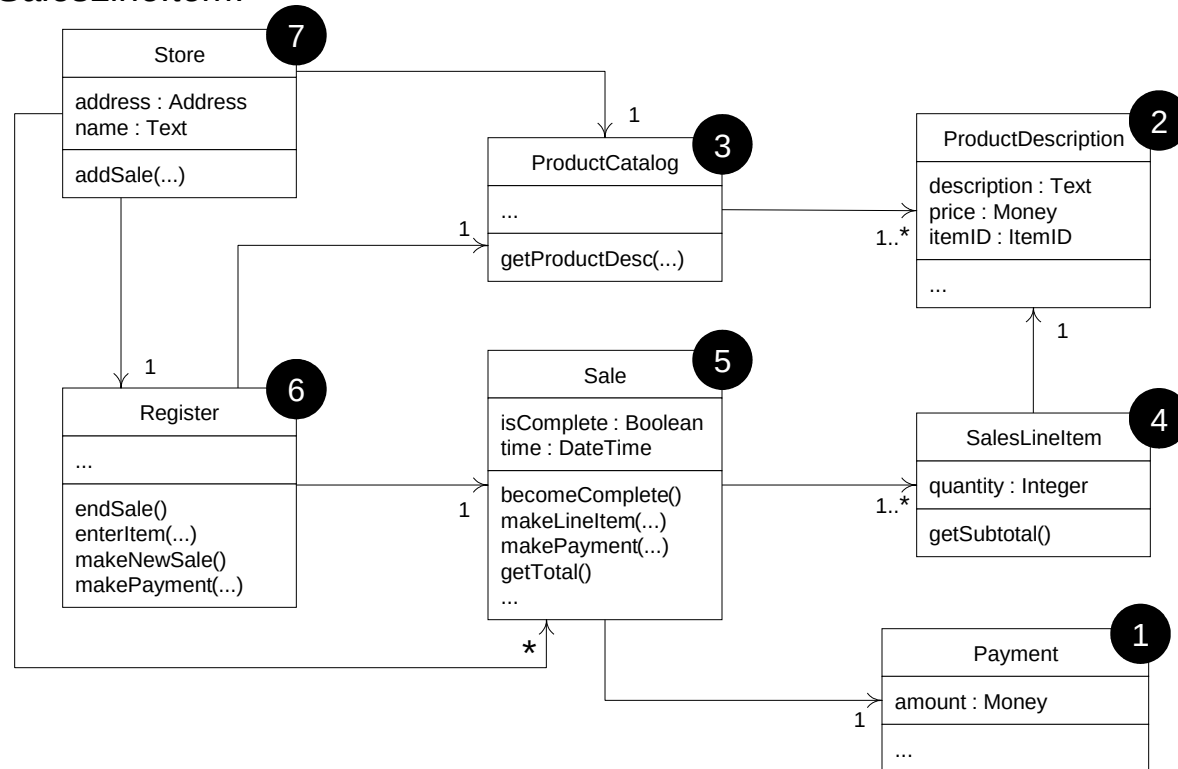
- One-to-many relationships are common.
 - For example, a *Sale* must maintain visibility to a group of many *SalesLineItem* instances.



- In OO programming languages, they are usually implemented with the introduction of a **collection object** of collection classes.
 - List (*ArrayList* – *List* interface) : a growing ordered list
 - Map (*HashMap* – *Map* interface) : a key-based lookup
 - Simple array

Order of Implementation

- Classes need to be implemented (and ideally, fully unit tested) from least-coupled to most-coupled. *→ 다른걸 몰라도 되는 클래스들 부터*
 - For example,
 - Possible first classes to implement are either *Payment* or *ProductDescription*.
 - Next are classes only dependent on the prior implementations; *ProductCatalog* or *SalesLineItem*.

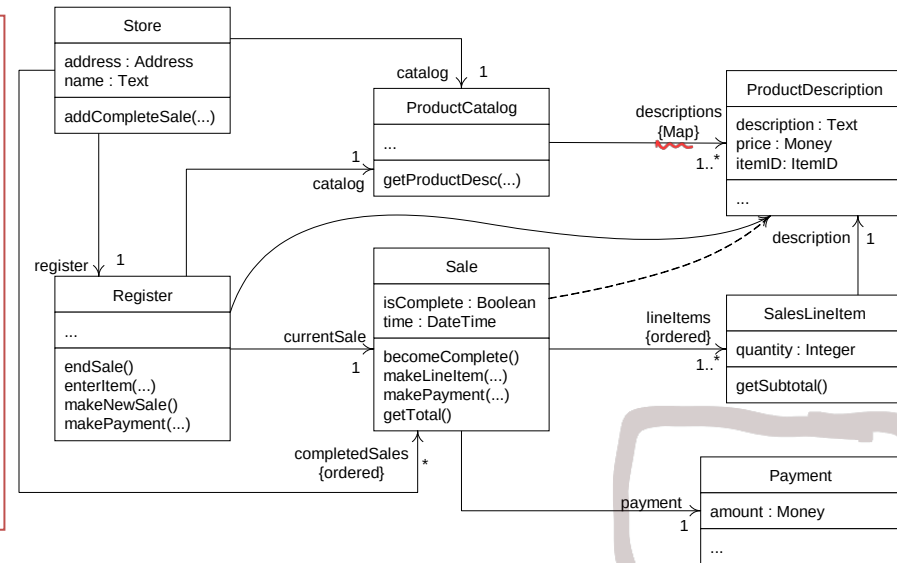


A possible order of class implementation and testing

Example: the NextGen POS Program Solution

- The main point of this listing is to show that there is a translation from design artifacts to a foundation of code.
 - This code defines a simple case; it is not meant to illustrate a robust, fully developed Java program with synchronization, exception handling, and so on.

```
// all classes are probably in a package named something like:  
// package com.foo.nextgen.domain;  
  
public class Payment  
{  
    private Money amount;  
  
    public Payment( Money cashTendered ){ amount = cashTendered; }  
    public Money getAmount() { return amount; }  
}
```



```

public class ProductCatalog
{
    private Map<ItemID, ProductDescription>
        descriptions = new HashMap()<ItemID, ProductDescription>;

    public ProductCatalog()
    {
        // sample data
        ItemID id1 = new ItemID( 100 );
        ItemID id2 = new ItemID( 200 );
        Money price = new Money( 3 );

        ProductDescription desc;
        desc = new ProductDescription( id1, price, "product 1" );
        descriptions.put( id1, desc );
        desc = new ProductDescription( id2, price, "product 2" );
        descriptions.put( id2, desc );
    }

    public ProductDescription getProductDescription( ItemID id )
    {
        return descriptions.get( id );
    }
}

```

```

public class ProductDescription
{
    private ItemID id; private Money price;
    private String description;

    public ProductDescription ( ItemID id, Money price, String description )
    {
        this.id = id; this.price = price;
        this.description = description;
    }

    public ItemID getItemID() { return id; }

    public Money getPrice() { return price; }
    public String getDescription() { return description; }
}

```

```

public class Sale
{
    private List<SalesLineItem> lineItems = new ArrayList<>(<SalesLineItem>);
    private Payment payment;

    private Date date = new Date();
    private boolean isComplete = false;

    public Money getBalance()
    {
        return payment.getAmount().minus( getTotal() );
    }

    public void becomeComplete() { isComplete = true; }

    public boolean isComplete() { return isComplete; }

    public void makeLineItem( ProductDescription desc, int quantity )
    {
        lineItems.add( new SalesLineItem( desc, quantity ) );
    }

    public Money getTotal()
    {
        Money total = new Money();  Money subtotal = null;

        for ( SalesLineItem lineItem : lineItems )
        {
            subtotal = lineItem.getSubtotal();  total.add( subtotal );
        }
        return total;
    }

    public void makePayment( Money cashTendered )
    {
        payment = new Payment( cashTendered );
    }
}

```

```
public class SalesLineItem
{
    private int quantity;
    private ProductDescription description;

    public SalesLineItem (ProductDescription desc, int quantity )
    {
        this.description = desc;  this.quantity = quantity;
    }

    public Money getSubtotal()
    {
        return description.getPrice().times( quantity );
    }
}
```

```
public class Store
{
    private ProductCatalog catalog = new ProductCatalog();
    private Register register = new Register( catalog );

    public Register getRegister() { return register; }
}
```